

Student

I've seen colons and arrows in Python function signatures like `def greet(name: str) → str:` but I always ignored them. What are they actually doing?

ChatGPT • AI Assistant

CONCEPT CARD

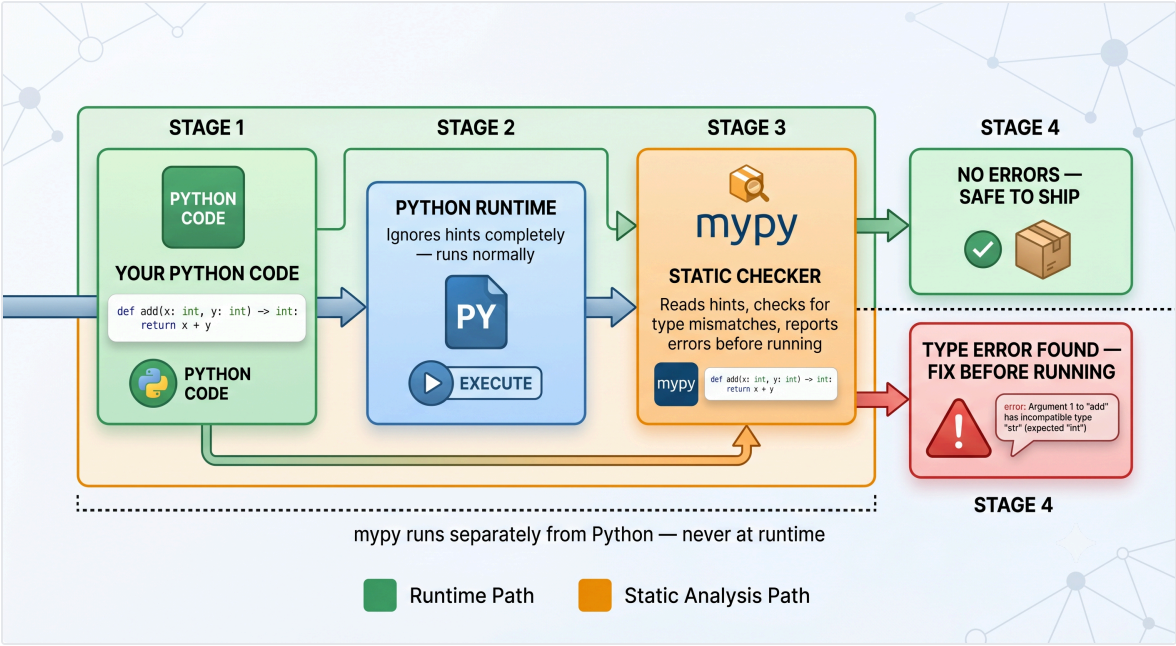
Those are type hints — one of the most powerful tools for writing clear, safe Python. Here's the key insight that surprises most beginners: Python completely ignores type hints at runtime. They do not change how your code executes. They exist purely for two audiences: tools (like mypy, your IDE) and humans (your teammates, your future self).

🗨️ ANALOGY

Think of type hints like road signs. A 'Speed Limit 60' sign does not physically stop your car. But it tells you — and a traffic camera — what the expected behavior is. Type hints are the same: they tell mypy and your IDE what types are expected, and those tools will catch violations before your code ever runs.

SUMMARY CARD

- Type hints are checked by tools, not by Python itself
- mypy is the most popular static type checker for Python
- Benefits: catch bugs before runtime, better IDE autocomplete, self-documenting code
- Type hints were introduced in PEP 484 (Python 3.5) and have evolved significantly



Type Hints Ecosystem — From Code to Static Analysis

Type Hint Quick Reference — Most Common Annotations

Annotation	Meaning	Example Usage	Python Version
<code>str, int, float, bool</code>	Built-in primitive types	<code>name: str = 'Alice'</code>	3.0+
<code>list[str]</code>	List of strings	<code>tags: list[str] = []</code>	3.9+
<code>dict[str, int]</code>	Dict with str keys, int values	<code>scores: dict[str, int] = {}</code>	3.9+
<code>tuple[int, str, bool]</code>	Fixed-length tuple	<code>record: tuple[int, str, bool]</code>	3.9+
<code>Optional[str]</code>	str or None	<code>middle_name: Optional[str] = None</code>	3.5+
<code>str None</code>	str or None (modern syntax)	<code>middle_name: str None = None</code>	3.10+
<code>Union[str, int]</code>	str or int	<code>id: Union[str, int]</code>	3.5+
<code>Any</code>	Disables type checking	<code>data: Any</code>	3.5+

Annotation	Meaning	Example Usage	Python Version
Callable[[int, str], bool]	Function taking int,str returning bool	validator: Callable[[int, str], bool]	3.5+
TypeVar	Generic type placeholder	T = TypeVar('T')	3.5+

BASIC TYPE ANNOTATIONS — VARIABLES AND FUNCTIONS

ChatGPT • AI Assistant

A detailed labeled diagram of a single Python function definition. The function shown is: `def calculate_discount(price: float, discount_pct: float, label: str) -> str:` Each part is annotated with a colored callout arrow. Arrow 1 (blue) points to 'price' labeled 'Parameter name'. Arrow 2 (orange) points to ': float' labeled 'Type annotation — after colon'. Arrow 3 (green) points to '-> str' labeled 'Return type annotation — after arrow'. Arrow 4 (purple) points to the default value area labeled 'Default values come after the type annotation: label: str = "Item"'. Below the function, a gray box labeled 'At Runtime' shows: Python reads name and default — IGNORES the type annotations completely. A second box labeled 'mypy sees' shows: Checks all call sites to verify float, float, str are passed and str is returned.

Anatomy of a Type-Annotated Python Function

Student

Let's start simple. How do I add type hints to a basic function?

ChatGPT • AI Assistant

Type hints go in two places: after the parameter name with a colon, and after the closing parenthesis with an arrow for the return type.

EXAMPLE CARD

Example 1 — Basic: No hints vs with hints:

python — Code Example

```
# Without type hints — what types are these?
def calculate_discount(price, discount_pct, label):
```

```

discounted = price * (1 - discount_pct / 100)
return f"{label}: ${discounted:.2f}"

# With type hints — immediately clear
def calculate_discount(price: float, discount_pct: float, label: str)
→ str:
    discounted = price * (1 - discount_pct / 100)
    return f"{label}: ${discounted:.2f}"

result = calculate_discount(100.0, 10.0, "Sale Price")
print(result)
# Output: Sale Price: $90.00

```

Student

What if a function returns nothing?

ChatGPT • AI Assistant

Use → None. This is important — it explicitly tells readers and mypy that this function has no meaningful return value.

Example 2 — Intermediate: Annotating variables and None returns:

python — Code Example

```

from typing import Optional

# Variable annotations
user_name: str = "Alice"
age: int = 30
active: bool = True
balance: float = 0.0

# Function returning None
def log_event(event: str, level: str = "INFO") → None:
    print(f"[{level}] {event}")

# Optional parameter — might be None
def find_user(user_id: int) → Optional[str]:
    users = {1: "Alice", 2: "Bob"}
    return users.get(user_id) # Returns str or None

```



```
print(find_user(1))    # Output: Alice
print(find_user(99))   # Output: None
```

Example 3 — Practical: Annotating a data processing function:

python – Code Example

```
def parse_age(value: str) → int:
    """Convert string input to integer age."""
    return int(value.strip())

def format_user_record(name: str, age: int, email: str) → dict[str,
str | int]:
    """Build a user record dictionary."""
    return {
        "name": name,
        "age": age,
        "email": email,
        "status": "active"
    }

record = format_user_record("Alice", 30, "alice@example.com")
print(record)
# Output: {'name': 'Alice', 'age': 30, 'email': 'alice@example.com',
'status': 'active'}
```

❗ COMMON PITFALL / MISTAKE

Incorrect Way:

python – Code Example

```
def get_score() → int:
    return None    # mypy error! None is not int
```

Correct Way:

python – Code Example

```
def get_score() → Optional[int]:
    return None    # Explicitly declare it might return None
```

EXAM TIP CARD

The `→ None` return annotation is not optional — always include it on functions that return nothing. `mypy` will infer it anyway but explicit is better than implicit (PEP 20).

INTERVIEW CARD

Frequency: Very Common

Question: 'What are type hints in Python and do they affect runtime performance?'

Expected Answer: Type hints are annotations added to function parameters and return values that describe expected types. Python completely ignores them at runtime — they have zero performance impact. They are used by static analysis tools like `mypy` and IDEs to catch type errors before execution and improve code readability.

Follow-up: 'What is the difference between `Optional[str]` and `str | None`'?

Expected Answer: They are equivalent. `Optional[str]` is the older syntax from typing module. `str | None` is the modern Python 3.10+ union syntax using the `|` operator. Both mean 'this value is either a string or None'.

EXERCISE

Add complete type hints to this untyped function:

python — Code Example

```
def process_order(order_id, items, apply_tax):
    total = sum(item['price'] * item['qty'] for item in items)
    if apply_tax:
        total = total * 1.18
    return {'order_id': order_id, 'total': round(total, 2)}
```

Hint: `items` is a list of dicts with `str` keys and mixed values.

AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

'Act as a senior Python mentor specializing in type safety. Give me 10 progressively harder functions — start with simple `str/int` annotations and build up to `Optional`, `Union`, and `Callable`. For each function: show me the untyped version first, ask me to write the typed version, then reveal the correct answer and explain any mistakes I might make. Include one tricky example involving `None` handling that beginners always get wrong.'

🔗 PRACTICE QUIZ

1. Do type hints affect Python's runtime behavior?
2. What annotation should a function use if it returns nothing?
3. Predict: `def greet(name: str) → str: return 42`. Does Python raise an error when you call `greet('Alice')`?
4. What is the difference between `Optional[str]` and `str`?
5. Write the type hint for a function that takes a list of integers and returns a float.
6. Debug: mypy reports 'Incompatible return value type (got int, expected str)' on `return len(text)`. The function signature is `→ str`. How do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. No — Python ignores type hints completely at runtime
2. `→ None`
3. No! Python does not enforce type hints at runtime — it returns 42 silently. Only mypy catches this.
4. `Optional[str]` means the value can be `str` OR `None`. `str` alone means only `str` is accepted — `None` would be a type error.
5. `def sum_values(numbers: list[int]) → float:`
6. Change return type to `→ int`, or wrap with `return str(len(text))` if string is truly needed

🔗 CHECKPOINT QUESTION

If type hints don't affect runtime, why bother writing them at all?

CONTAINER TYPES, UNION, AND CALLABLE ANNOTATIONS

ChatGPT • AI Assistant

A tree diagram showing Python container type annotations organized by complexity. Root node labeled 'Container Types'. Three main branches: (1) Sequence branch: `list[T] → list[str], list[int], list[dict[str,int]]`; (2) Mapping branch: `dict[K, V] → dict[str, str], dict[str, int], dict[str, list[str]]`; (3) Function branch: `Callable[[Args], Return] → Callable[[int], str], Callable[[str, int], bool], Callable[..., None]`. Each leaf node shows a concrete example and a small code snippet. Color coding: list nodes in green, dict nodes in blue, Callable nodes in purple. A side note on the right reads: 'Python 3.9+: use built-in `list[]`, `dict[]`, `tuple[]` directly. Python 3.5-3.8: import `List`, `Dict`, `Tuple` from `typing` module'.

Container Type Hierarchy in Python Type Hints

Student

Basic types make sense, but what about annotating lists, dicts, and functions that take other functions as arguments?

ChatGPT • AI Assistant

Container types let you specify not just that something is a list, but what kind of list. This is where type hints become genuinely powerful — mypy can catch errors like passing a list of strings where a list of integers is expected.

EXAMPLE CARD

Example 1 — Basic container types:

python — Code Example

```
# list[str] — a list containing only strings
def get_usernames(user_ids: list[int]) → list[str]:
    db = {1: "alice", 2: "bob", 3: "carol"}
    return [db[uid] for uid in user_ids if uid in db]

result = get_usernames([1, 3])
print(result) # Output: ['alice', 'carol']

# dict[str, int] — keys are strings, values are integers
def count_words(text: str) → dict[str, int]:
    counts: dict[str, int] = {}
    for word in text.lower().split():
        counts[word] = counts.get(word, 0) + 1
    return counts

print(count_words("hello world hello"))
# Output: {'hello': 2, 'world': 1}
```

Example 2 — Intermediate: tuple and nested containers:

python — Code Example

```

# tuple with fixed types
def get_coordinates() → tuple[float, float]:
    return (37.7749, -122.4194)

lat, lon = get_coordinates()
print(f"Lat: {lat}, Lon: {lon}")
# Output: Lat: 37.7749, Lon: -122.4194

# list of dicts — common in API responses
def get_users() → list[dict[str, str | int]]:
    return [
        {"name": "Alice", "age": 30},
        {"name": "Bob", "age": 25}
    ]

```

Example 3 — Practical: Callable annotations for higher-order functions:

python — Code Example

```

from typing import Callable

# Callable[[ArgType1, ArgType2], ReturnType]
def apply_transform(
    values: list[int],
    transform: Callable[[int], int]
) → list[int]:
    return [transform(v) for v in values]

def double(x: int) → int:
    return x * 2

result = apply_transform([1, 2, 3, 4], double)
print(result) # Output: [2, 4, 6, 8]

# Lambda also works — mypy infers the type
result2 = apply_transform([1, 2, 3], lambda x: x ** 2)
print(result2) # Output: [1, 4, 9]

```

Example 4 — Real Business Example: Processing API response data:

python — Code Example

```

from typing import Callable, Optional

# Representing a pipeline of data transforms
def build_pipeline(
    steps: list[Callable[[dict[str, str | int]], dict[str, str | int]]]
) → Callable[[dict[str, str | int]], dict[str, str | int]]:
    def run(data: dict[str, str | int]) → dict[str, str | int]:
        for step in steps:
            data = step(data)
        return data
    return run

def normalize_name(record: dict[str, str | int]) → dict[str, str | int]:
    record["name"] = str(record["name"]).strip().title()
    return record

def add_status(record: dict[str, str | int]) → dict[str, str | int]:
    record["status"] = "active"
    return record

pipeline = build_pipeline([normalize_name, add_status])
result = pipeline({"name": "  alice  ", "age": 30})
print(result)
# Output: {'name': 'Alice', 'age': 30, 'status': 'active'}

```

Student

This is getting complex. When should I use Any?

ChatGPT • AI Assistant

⚠ WARNING

Any is an escape hatch — it tells mypy to stop checking that value entirely. Use it only when:

- Integrating with untyped third-party libraries
- You genuinely cannot express the type statically

Never use Any just to avoid thinking about types — it defeats the entire purpose of type hints.

❗ COMMON PITFALL / MISTAKE

Incorrect Way:

python – Code Example

```
def process(data: Any) → Any: # Turns off all checking  
    return data['name'].upper()
```

Correct Way:

python – Code Example

```
def process(data: dict[str, str]) → str: # mypy can now verify  
everything  
    return data['name'].upper()
```

🔗 INTERVIEW CARD

Frequency: Moderate

Question: 'How do you annotate a function that accepts another function as an argument?'

Expected Answer: Use `Callable[[ArgTypes], ReturnType]` from the typing module. For example, `Callable[[int, str], bool]` means a function that takes an int and a str and returns a bool. In Python 3.9+ you can also use `collections.abc.Callable`.

Follow-up: 'What does `Callable[..., str]` mean?'

Expected Answer: The ellipsis ... means the callable can take any arguments — only the return type str is constrained. Useful when you only care about the return type.

🔥 EXERCISE

Write type annotations for these functions:

1. A function that takes a list of (name, score) tuples and returns a dict mapping names to scores
2. A function that takes a filter function and a list of strings, and returns filtered strings
3. A function that takes a dict of student names to their list of grades, and returns the student with the highest average

🌟 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

'Act as a mypy expert. Give me 8 progressively harder type annotation challenges involving containers: start with `list[str]`, then `dict[str, list[int]]`, then nested dicts, then Callable with containers, then Optional containers. For each: show me the untyped function, ask me to annotate it, reveal the answer, and show what error mypy would catch without the annotation.'

🔗 PRACTICE QUIZ

1. What is the type hint for a list of floats?
2. What does `Callable[[int, str], bool]` mean?
3. Predict: `def f(items: list[int]) → None: items.append('hello')`. Does Python raise an error at runtime?
4. What is the difference between `tuple[int, int]` and `tuple[int, ...]`?
5. When is it acceptable to use `Any`?
6. Debug: mypy says 'List item 0 has incompatible type `str`; expected `int`' on line `return ["error", "ok"]`. The return type is `list[int]`. How do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `list[float]`
2. A callable (function) that takes an `int` and a `str` as arguments and returns a `bool`
3. No! Python does not enforce `list[int]` at runtime — 'hello' is appended without error. Only mypy catches this.
4. `tuple[int, int]` is a fixed 2-element tuple of ints. `tuple[int, ...]` is a variable-length tuple where all elements are ints.
5. Only when integrating with untyped libraries or when the type is genuinely dynamic and cannot be expressed statically
6. Change the return type to `list[str]`, or change the returned strings to integers

🔗 CHECKPOINT QUESTION

Write the type hint for a function that accepts a list of dictionaries (each with string keys and integer values) and returns a single dictionary with the same structure.

TYPEVAR AND GENERIC FUNCTIONS

ChatGPT • AI Assistant

A diagram showing a single generic function definition at the top: `T = TypeVar('T') / def first(items: list[T]) → T`. Below it, three call site boxes connected by arrows. Call Site 1: `first([1, 2, 3])` — arrow labeled '`T resolves to int`' pointing to box showing '`Return type: int`'. Call Site 2: `first(['a', 'b', 'c'])` — arrow labeled '`T resolves to str`' pointing to box showing '`Return type: str`'. Call Site 3: `first([3.14, 2.71])` — arrow labeled '`T resolves to float`' pointing to box showing '`Return type: float`'. Each resolution box has a checkmark and a green border. A separate red box below shows: `first([])` — arrow labeled '`Cannot resolve T — mypy warns about empty list`'. The figure communicates that TypeVar is resolved independently at each call site, not globally.

How TypeVar Resolves at Each Call Site

Student

What if I want to write a function that works with any type but still preserves type safety? Like a function that returns the same type it receives?

ChatGPT • AI Assistant

CONCEPT CARD

That is exactly what TypeVar solves. A TypeVar is a placeholder that says 'I don't know the specific type yet, but whatever goes in must match what comes out.' This is how you write generic functions that are both flexible and type-safe.

Without TypeVar, you'd be forced to use Any (which disables checking) or write separate functions for each type. TypeVar lets you write one function that mypy can still verify correctly for each call site.

EXAMPLE CARD

Example 1 — Basic: The classic identity problem:

python — Code Example

```
from typing import TypeVar

# Without TypeVar — any input, any output — no safety
def first_element_unsafe(items: list) → object:
    return items[0]

result = first_element_unsafe([1, 2, 3])
# mypy doesn't know result is int — can't do result + 1 safely
```

```
# With TypeVar – input type is preserved in output
T = TypeVar('T')

def first_element(items: list[T]) → T:
    return items[0]

num = first_element([1, 2, 3])
print(num + 1)  # Output: 2 – mypy knows num is int!

name = first_element(["alice", "bob"])
print(name.upper())  # Output: ALICE – mypy knows name is str!
```

Example 2 — Intermediate: Constrained `TypeVar`:

python – Code Example

```
from typing import TypeVar

# TypeVar constrained to specific types only
Numeric = TypeVar('Numeric', int, float)

def double(value: Numeric) → Numeric:
    return value * 2

print(double(5))      # Output: 10 (int)
print(double(3.14))   # Output: 6.28 (float)
# double("hello")    # mypy error! str is not int or float
```

Example 3 — Practical: Generic pair utility:

python – Code Example

```
from typing import TypeVar

K = TypeVar('K')
V = TypeVar('V')

def swap_pair(pair: tuple[K, V]) → tuple[V, K]:
    first, second = pair
    return (second, first)

result1 = swap_pair((1, "hello"))  # Returns tuple[str, int]
result2 = swap_pair(("x", 3.14))    # Returns tuple[float, str]
```

```
print(result1) # Output: ('hello', 1)
print(result2) # Output: (3.14, 'x')
```

Student

Is TypeVar commonly tested in interviews?

ChatGPT • AI Assistant

 INTERVIEW CARD

Frequency: Moderate — asked for mid-to-senior roles, especially at companies using strict mypy or in library development

Question: 'What is a TypeVar in Python and when would you use it?'

Expected Answer: TypeVar is a generic type variable from the typing module. It lets you write functions that preserve type relationships between inputs and outputs without losing type information. For example, a function that takes a list[T] and returns T tells mypy that the return type is the same as the list element type — enabling full type checking at each call site.

Follow-up: 'What is the difference between T = TypeVar("T") and T = TypeVar("T", int, float)?'

Expected Answer: Without constraints, T can be any type. With constraints (int, float), T can only be int or float — mypy will reject any other type.

 COMMON PITFALL / MISTAKE

Incorrect Way:

python — Code Example

```
T = TypeVar('T')
def process(x: T) → T:
    return x + 1 # mypy error! + is not valid for all types T could
be
```

Correct Way:

python — Code Example

```
Numeric = TypeVar('Numeric', int, float)
def increment(x: Numeric) → Numeric:
```

```
return x + 1 # Safe – both int and float support +
```

🔥 EXERCISE

Write a generic function using `TypeVar` that:

1. Takes two lists of the same type `T` and returns a merged list of type `list[T]`
2. Takes a value of type `T` and a `list[T]` and returns `True` if the value is in the list

🌟 AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

'Act as a Python generics expert. Teach me `TypeVar` from first principles. Start with the problem it solves (loss of type info with `Any` or `object`), then show me 6 progressively harder examples of `TypeVar` usage. Include: basic identity, constrained `TypeVar`, multiple `TypeVars` in one function, `TypeVar` in a class method. Ask me to predict the mypy output for each example.'

🔍 PRACTICE QUIZ

1. What does `TypeVar('T')` represent?
2. Can a constrained `TypeVar` like `TypeVar('N', int, float)` accept a `str` argument?
3. Predict: `T = TypeVar('T') / def identity(x: T) → T: return 42`. What does mypy say?
4. Why is `TypeVar` better than `Any` for generic functions?
5. What import is needed to use `TypeVar`?
6. Debug: A function `def wrap(x: T) → list[T]` returns `[x, x]`. mypy says 'Name `T` is not defined'. What is missing?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. A type variable — a placeholder that will be resolved to a specific type at each call site
2. No — mypy will report an error. Only `int` and `float` are allowed.
3. mypy error: 'Incompatible return value type (got `int`, expected `T`)' — you promised to return `T` but returned a literal `int`

4. Any disables all type checking on that value everywhere. TypeVar preserves the type relationship so mypy can still verify correctness.
5. from typing import TypeVar
6. T = TypeVar('T') is missing — TypeVar must be declared before use

🔗 CHECKPOINT QUESTION

In your own words, what problem does TypeVar solve that you cannot solve with just using Any?

TYPEDDICT AND DATACLASS TYPE INTERACTIONS

ChatGPT • AI Assistant

A side-by-side comparison diagram with two columns. Left column header: 'TypedDict' with a blue background. Shows a TypedDict definition with fields id: int, name: str, email: str. Below it, an access example using dict-style: user['name']. A list of characteristics: 'Is a plain dict at runtime', 'No methods', 'Dict-style access', 'Great for JSON/API data', 'mypy checks key names and types'. Right column header: 'dataclass' with a green background. Shows a @dataclass definition with the same fields plus a method display() → str. Below it, an access example using attribute-style: user.name. A list of characteristics: 'Creates a real class', 'Can have methods', 'Attribute-style access', 'Great for domain objects', '__init__', '__repr__', '__eq__' auto-generated. A center divider with a question: 'External data? → TypedDict. Internal domain model? → dataclass'.

TypedDict vs dataclass — Structure and Access Comparison

Student

I often use plain dicts to pass data around — like {'name': 'Alice', 'age': 30}. Can I add type hints to those?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — that is exactly what TypedDict is for. A TypedDict lets you define a dictionary with specific required keys and their types. mypy will then verify that you always provide the right keys with the right value types.

This is extremely useful for API response types, configuration objects, and anywhere you use dicts as structured records. TypedDict gives you the flexibility of a dict with the safety of a class.

EXAMPLE CARD

Example 1 — Basic TypedDict:

python — Code Example

```
from typing import TypedDict

class UserRecord(TypedDict):
    name: str
    age: int
    email: str

# Correct usage — mypy is happy
def create_user(name: str, age: int, email: str) → UserRecord:
    return {"name": name, "age": age, "email": email}

user = create_user("Alice", 30, "alice@example.com")
print(user["name"]) # Output: Alice

# mypy catches this:
# return {"name": name, "age": "thirty", "email": email}
# Error: Value of key 'age' has type str, expected int
```

Example 2 — Intermediate: TypedDict with optional keys:

python — Code Example

```
from typing import TypedDict, Optional

class ProductRecord(TypedDict, total=False): # total=False makes all
    keys optional
    id: int
    name: str
    description: str
    price: float

# Or mix required and optional:
class OrderRecord(TypedDict):
    order_id: int
    customer: str

class FullOrderRecord(OrderRecord, total=False): # Extends with
    optional fields
    discount_code: str
    notes: str
```

```
order: FullOrderRecord = {"order_id": 101, "customer": "Bob"}  
print(order) # Output: {'order_id': 101, 'customer': 'Bob'}
```

Example 3 — Practical: TypedDict vs dataclass comparison:

python — Code Example

```
from typing import TypedDict  
from dataclasses import dataclass  
  
# TypedDict — stays a plain dict, good for JSON/API data  
class APIUser(TypedDict):  
    id: int  
    username: str  
    email: str  
  
# dataclass — becomes a real class with methods  
@dataclass  
class AppUser:  
    id: int  
    username: str  
    email: str  
  
    def display_name(self) → str:  
        return f"{self.username} <{self.email}>"  
  
# TypedDict usage — feels like dict  
api_data: APIUser = {"id": 1, "username": "alice", "email": "a@b.com"}  
print(api_data["username"]) # dict-style access  
  
# dataclass usage — feels like class  
app_user = AppUser(id=1, username="alice", email="a@b.com")  
print(app_user.display_name()) # Output: alice <a@b.com>  
print(app_user.username)      # attribute access
```

Student

When should I use TypedDict vs dataclass?

ChatGPT • AI Assistant

SUMMARY CARD

- Use TypedDict when: data comes from JSON/API, you need dict-style access (data['key']), you're working with existing dict-based code
- Use dataclass when: you need methods, you want attribute access (obj.field), you want auto-generated **init**, **repr**, **eq**
- Rule of thumb: external data (API responses) → TypedDict. Internal domain objects → dataclass.

❗ COMMON PITFALL / MISTAKE

Incorrect Way:

python – Code Example

```
# Using a plain dict and hoping everyone remembers the structure
def process(user: dict) → str: # What keys does user have? No idea!
    return user['name'].upper()
```

Correct Way:

python – Code Example

```
from typing import TypedDict
class User(TypedDict):
    name: str
    age: int

def process(user: User) → str: # Now mypy knows exactly what keys exist
    return user['name'].upper()
```

📄 INTERVIEW CARD

Frequency: Moderate — asked especially in backend and data engineering roles

Question: 'What is the difference between TypedDict and dataclass in Python?'

Expected Answer: TypedDict defines a dictionary with a fixed schema — the resulting object is still a plain dict and accessed with dict['key'] syntax. dataclass creates a real class with attribute access, auto-generated methods (**init**, **repr**, **eq**), and can have methods. TypedDict is ideal for typed API/JSON data. dataclass is better for domain model objects that need behavior.

Follow-up: 'What does total=False do in a TypedDict?'

🔥 EXERCISE

1. Define a TypedDict called InvoiceItem with fields: sku (str), quantity (int), unit_price (float), discount (optional float)
2. Write a function that takes list[InvoiceItem] and returns the total cost as a float
3. Rewrite InvoiceItem as a dataclass and add a method line_total() → float

✧ AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

'Act as a Python architecture mentor. Walk me through 5 real-world scenarios — API client, database layer, config loader, event system, and CLI tool — and for each, explain whether TypedDict, dataclass, Pydantic model, or plain dict is the best choice. Show me the actual code for each and explain what bugs TypedDict or dataclass annotations would catch that a plain dict would miss.'

🔗 PRACTICE QUIZ

1. What is the main difference between a TypedDict and a regular dict at runtime?
2. What does total=False mean in a TypedDict definition?
3. Predict: class Config(TypedDict): host: str; port: int — you write config: Config = {"host": "localhost"}. What does mypy say?
4. Can a TypedDict have methods like a regular class?
5. What auto-generated methods does @dataclass provide?
6. Debug: You have class User(TypedDict): name: str; age: int and you write user['username'] — what error does mypy report and how do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. At runtime, they are both plain dicts — TypedDict adds no runtime behavior. The difference is only visible to mypy.
2. All keys become optional — you can omit any of them without a mypy error
3. mypy error: 'Missing key port' — total defaults to True, making all keys required
4. No — TypedDict is a dict, not a class. Methods are not supported.
5. **init**, **repr**, and **eq** are auto-generated. **hash** is added if frozen=True.

6. mypy reports 'TypedDict User has no key username'. Fix: either add username: str to the TypedDict, or change the access to user['name'].

🔍 CHECKPOINT QUESTION

You're building a function that parses a JSON API response with a known structure. Would you use TypedDict or dataclass for the response type, and why?

RUNNING MYPY — CATCHING REAL BUGS BEFORE RUNTIME

ChatGPT • AI Assistant

A circular workflow diagram showing the development cycle with mypy integrated. Five stages arranged in a circle with arrows connecting them clockwise: Stage 1 (blue): 'Write Code' — icon of a code editor. Stage 2 (orange): 'Add Type Hints' — icon of annotation arrows. Stage 3 (purple): 'Run mypy' — icon of a terminal window with mypy output. Stage 4 (red): 'Fix Type Errors' — icon of a wrench with error messages. Stage 5 (green): 'Commit + CI Check' — icon of a git branch with a green checkmark. A side branch from Stage 3 points to a box labeled 'mypy catches: wrong arg types, unhandled None, wrong return types, missing keys'. Below the circle, a comparison box shows two columns: 'Without mypy — bugs found in production' vs 'With mypy — bugs found before commit'. Green arrows indicate the mypy path is faster and safer.

mypy in the Development Workflow

Student

I've learned the syntax, but how do I actually run mypy and understand what it tells me?

ChatGPT • AI Assistant

mypy is a separate command-line tool. It reads your Python files, analyzes all type annotations, and reports errors without running the code. This means it can catch bugs that would only appear at runtime — sometimes in production — with real user data.

EXAMPLE CARD

Example 1 — Installation and basic run:

bash — Code Example

```
# Install mypy
pip install mypy
```

```
# Run on a single file
mypy app.py

# Run on a whole package
mypy src/

# Strict mode – enables all checks
mypy --strict app.py
```

Example 2 — Real before/after: ETL utility function:

python – Code Example

```
# BEFORE – No type hints (20-line ETL function)
def process_user_records(records, default_role, active_only):
    results = []
    for record in records:
        if active_only and not record.get('active'):
            continue
        user = {
            'id': record['id'],
            'name': record['name'].strip(),
            'role': record.get('role') or default_role,
            'score': record['score'] * 1.1
        }
        results.append(user)
    return results

# mypy on untyped code: 'note: By default, mypy ignores untyped code'
# You catch NOTHING without annotations
```

python – Code Example

```
# AFTER – Fully annotated
from typing import TypedDict, Optional

class RawRecord(TypedDict):
    id: int
    name: str
    score: float
    active: bool
    role: Optional[str]

class ProcessedUser(TypedDict):
    id: int
```

```

    name: str
    role: str
    score: float

def process_user_records(
    records: list[RawRecord],
    default_role: str,
    active_only: bool
) → list[ProcessedUser]:
    results: list[ProcessedUser] = []
    for record in records:
        if active_only and not record['active']:
            continue
        user: ProcessedUser = {
            'id': record['id'],
            'name': record['name'].strip(),
            'role': record['role'] or default_role,
            'score': record['score'] * 1.1
        }
        results.append(user)
    return results

```

Now mypy catches:

1. If caller passes score as str instead of float

2. If record['role'] is accessed without handling None

3. If return value is used as wrong type downstream

Example 3 — Understanding mypy error messages:

code — Code Example

mypy output examples:

```

app.py:14: error: Argument 1 to "process_user_records" has
incompatible type
    "list[dict[str, str]]"; expected "list[RawRecord]"

```

```

app.py:22: error: Item "None" of "Optional[str]" has no attribute
"upper"

```

```

app.py:31: error: Incompatible return value type
    (got "list[ProcessedUser]", expected "None")

```

Reading mypy errors:

```
# filename:line_number: error: description
# The description tells you EXACTLY what type was found vs expected
```

Student

What about # type: ignore? I see it in codebases sometimes.

ChatGPT • AI Assistant

⚠ WARNING

type: ignore is an escape hatch that tells mypy to skip checking that line. Use it only when:

- The error is a known mypy limitation or bug
- You're integrating with a library that has no type stubs
- You have a very unusual dynamic pattern that cannot be statically typed

Never use it to 'silence' an error you don't understand — investigate the error first.

python — Code Example

```
# Acceptable use — third-party library with no stubs
import some_untyped_lib # type: ignore[import-untyped]

# BAD use — silencing a real type error you should fix
result = process(data) # type: ignore # Never do this without a
comment explaining why
```

MEMORY TRICK CARD

Remember the mypy workflow as: WRITE → ANNOTATE → CHECK → FIX:

1. WRITE your function normally
2. ANNOTATE all parameters and return types
3. CHECK with mypy
4. FIX reported errors — they are real bugs!

🌐 REAL WORLD APPLICATION

At major tech companies, CI/CD pipelines run mypy --strict on every pull request. A PR cannot be merged if mypy reports errors. This catches type-related bugs before they reach code review — saving hours of debugging.

INTERVIEW CARD

Frequency: Common for senior roles

Question: 'What kinds of bugs does mypy catch that unit tests typically miss?'

Expected Answer: mypy catches: (1) functions called with wrong argument types, (2) None values not being handled before attribute access, (3) wrong return types, (4) accessing keys that don't exist in TypedDict, (5) passing a list[str] where list[int] is expected. Unit tests only catch these bugs if you wrote a test specifically for that wrong-type case — mypy catches them exhaustively across all code paths.

Follow-up: 'What is the difference between mypy --strict and running mypy without flags?'

EXERCISE

Download and run mypy on this buggy file:

python — Code Example

```
from typing import Optional

def get_user_email(user_id: int) → str:
    users = {1: {"name": "Alice", "email": "alice@example.com"}}
    user = users.get(user_id)
    return user["email"] # Bug: user might be None!

def calculate_total(prices: list[str]) → float:
    return sum(prices) # Bug: sum expects numbers, not strings

def greet(name: Optional[str]) → str:
    return "Hello " + name # Bug: name might be None!
```

Run mypy on this file. How many errors does it find? Fix all of them.

AI DEEP DIVE PROMPT

Copy this into Claude or ChatGPT:

'Act as a mypy expert and code reviewer. Take this Python function [paste a 20-line function without type hints]. First, identify 5 potential type-related bugs a human reviewer might miss. Then walk me through adding complete type annotations step by step. Finally, simulate what mypy --strict would report and explain each error message in plain language.'

🔗 PRACTICE QUIZ

1. What command runs mypy on a file called utils.py?
2. What does mypy --strict enable that regular mypy does not?
3. Predict: `def f(x: int) → str: return x`. What does mypy report?
4. When is `# type: ignore` acceptable to use?
5. What does this mypy error mean: `'Item None of Optional[str] has no attribute upper'`?
6. Debug: mypy reports 0 errors on a file full of untyped functions. Why might this be misleading, and how do you fix it?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. `mypy utils.py`
2. Strict mode enables checks like: disallowing untyped functions, checking return types even when inferred, requiring annotations on all public APIs
3. `error: Incompatible return value type (got int, expected str)`
4. When integrating with untyped third-party libraries, for known mypy limitations, or for genuinely untypeable dynamic patterns — always with an explanatory comment
5. You are calling `.upper()` on a value that is `Optional[str]`. Before calling `.upper()`, you must check it is not `None`: `if value is not None: value.upper()`
6. mypy by default ignores untyped functions — it reports no errors because it has nothing to check. Fix with `mypy --strict` or `mypy --disallow-untyped-defs` to force annotations on all functions

🔗 CHECKPOINT QUESTION

You have a 200-line Python module with no type hints and you want to gradually add mypy. What is the safest approach to start without breaking your workflow?

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including type hints and mypy, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Type Hints Ecosystem — From Code to Static Analysis:** A horizontal pipeline diagram with four stages connected by arrows. Stage 1 (gre...